

Informe Laboratorio 3 crypto y seguridad en redes

Análisis de Implementación de Algoritmos Hash en Autenticación Web

Seccion 4

Alumno: Diego Villagrán
e-mail: diego.villagran1@mail.udp.cl

17 de octubre de 2025

Índice

1. Descripción de Actividades	3
1.1. Metodología	3
1.2. Búsqueda del Sitio Objetivo	3
2. Desarrollo de Actividades	4
2.1. Identificación del algoritmo de hash en el registro	4
2.2. Identificación del algoritmo de hash al iniciar sesión	5
2.3. Generación del hash desde la consola del navegador	7
2.4. Interceptación del tráfico de login con BurpSuite	8
2.5. Intento de ataque Pass the Hash	9
2.6. Políticas de privacidad y seguridad	11
2.7. Conclusiones sobre la seguridad del sitio	12
2.7.1. Conclusión 1: Uso de SHA-1, un algoritmo obsoleto	12
2.7.2. Conclusión 2: Hashing del lado del cliente como falsa sensación de seguridad	12
2.7.3. Conclusión 3: Protección parcial contra Pass the Hash mediante limitación de caracteres	12
2.7.4. Conclusión 4: Falta de implementación de medidas de seguridad modernas	12
3. Referencias y Materiales	13

1. Descripción de Actividades

El objetivo de este laboratorio es evaluar la implementación de algoritmos hash aplicados a contraseñas en páginas web desde el lado del cliente, así como determinar la efectividad de estas medidas contra ataques de tipo Pass the Hash (PtH).

Para llevarlo a cabo, se realizó el registro en un sitio web "https://www.luckystarbus.com" creando una cuenta con una contraseña específica para las pruebas: ".^bc12345". Se analizó el comportamiento del sistema durante el proceso de registro e inicio de sesión, identificando el algoritmo de hash utilizado y evaluando posibles vulnerabilidades en su implementación.

1.1. Metodología

La metodología empleada incluyó las siguientes actividades:

1. Identificación del algoritmo de hash utilizado durante el registro
2. Identificación del algoritmo de hash utilizado durante el inicio de sesión
3. Generación del hash desde la consola del navegador
4. Interceptación del tráfico utilizando BurpSuite
5. Intento de ataque Pass the Hash
6. Análisis de políticas de privacidad y seguridad
7. Elaboración de conclusiones sobre la seguridad del sitio

1.2. Búsqueda del Sitio Objetivo

Para identificar un sitio web que implementara hashing de contraseñas del lado del cliente, se utilizó una consulta específica que permitió filtrar páginas con esta característica.

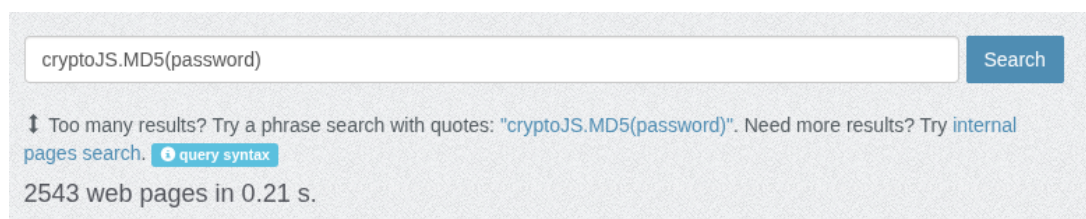
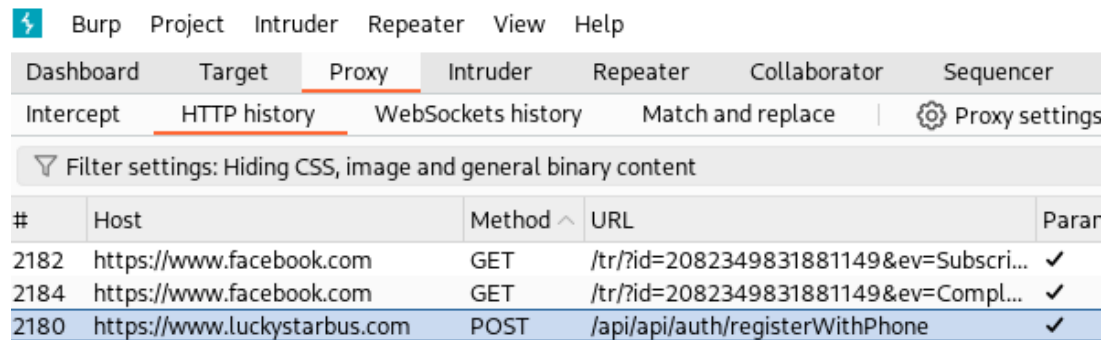


Figura 1: Query utilizada para encontrar páginas que hashean contraseñas: `CryptoJs.MD5(password)`

2. Desarrollo de Actividades

2.1. Identificación del algoritmo de hash en el registro

Durante el proceso de registro, se capturó el tráfico HTTP utilizando BurpSuite. Se identificó que el sitio utiliza el algoritmo **SHA-1** para hashear las contraseñas del lado del cliente antes de enviarlas al servidor.



The screenshot shows the Burp Suite interface with the 'HTTP history' tab selected. A table of captured requests is visible, with the third request (ID 2180) highlighted in blue. This request is a POST to the endpoint /api/api/auth/registerWithPhone on the domain https://www.luckystarbus.com.

#	Host	Method ^	URL	Parar
2182	https://www.facebook.com	GET	/tr?id=2082349831881149&ev=Subscri...	✓
2184	https://www.facebook.com	GET	/tr?id=2082349831881149&ev=Compl...	✓
2180	https://www.luckystarbus.com	POST	/api/api/auth/registerWithPhone	✓

Figura 2: Paquete POST capturado con BurpSuite hacia el endpoint `api/api/auth/registerwithphone`

```
1 POST /api/api/auth/registerWithPhone HTTP/1.1
2 Host: www.luckystarbus.com
3 Cookie: JSESSIONID=5B9C0CCD831F59BFDB1E077D7F59CAEC; user=
  %7B%22phone%22%3Anull%2C%22email%22%3Anull%2C%22sessionId%22%3A0%2C%22t
  2%3A%222025-10-15T04%3A24%3A55.861Z%22%2C%22account%22%3Anull%2C%22pure
  GA1.2.587691094.1760502296; _ga_J2CH62KYR9=GS2.1.s1760502296$ol$g0$t176
4 Content-Length: 202
5 Sec-Ch-Ua-Platform: "Linux"
6 Accept-Language: es-419,es;q=0.9
7 Sec-Ch-Ua: "Not=A?Brand";v="24", "Chromium";v="140"
8 Accesstoken: CgYdiWl302
9 Sec-Ch-Ua-Mobile: ?0
10 Tid: 252789338893455872
11 X-Requested-With: XMLHttpRequest
12 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML,
13 Accept: application/json, text/javascript, */*; q=0.01
14 Content-Type: application/json
15 Channelid: 268720169679920128
16 Sessionid: 0
17 Origin: https://www.luckystarbus.com
18 Sec-Fetch-Site: same-origin
19 Sec-Fetch-Mode: cors
20 Sec-Fetch-Dest: empty
21 Referer: https://www.luckystarbus.com/
22 Accept-Encoding: gzip, deflate, br
23 Priority: u=1, i
24 Connection: keep-alive
25
26 {
  "email": "",
  "password": "dca0a5afd0b457ee36f8862369c7fda58c162b25",
  "phone": "+13159080980",
  "purePhone": "3159080980",
  "countryCode": "+1",
  "validationCode": "429727",
  "subscribe": true,
  "timestamp": 1760502703498
}
```

Figura 3: POST enviado al servidor durante el registro, mostrando la contraseña hasheada con SHA-1

El análisis del tráfico reveló que la contraseña es procesada mediante SHA-1 antes de ser transmitida, generando un hash de 40 caracteres hexadecimales característico de este algoritmo.

2.2. Identificación del algoritmo de hash al iniciar sesión

Para verificar la consistencia del algoritmo utilizado, se cerró la sesión y se realizó un nuevo inicio de sesión. Se capturó nuevamente el tráfico para confirmar el algoritmo empleado.

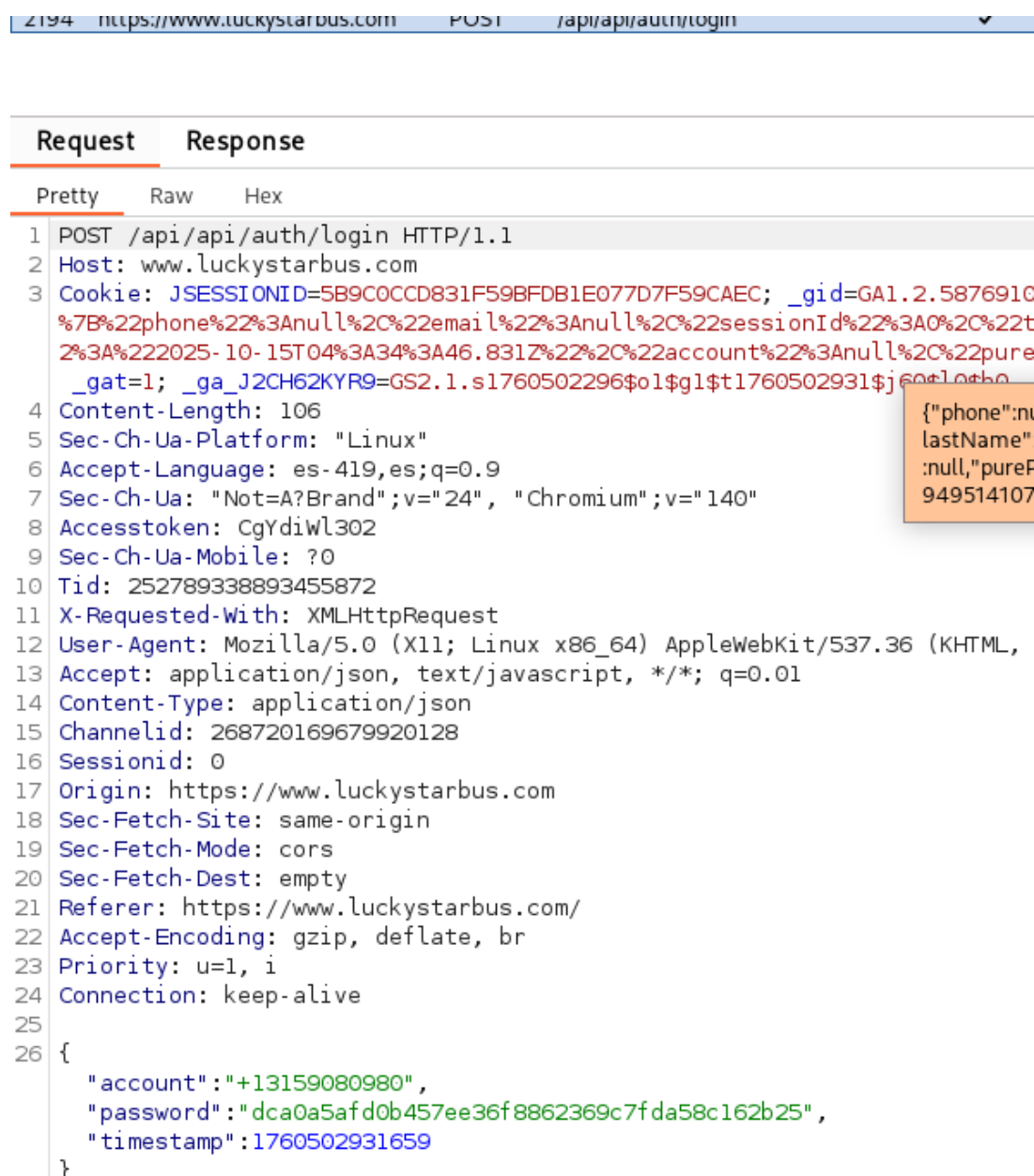


Figura 4: POST enviado al servidor durante el login, mostrando el mismo hash SHA-1 para la contraseña

Se confirmó que el mismo algoritmo **SHA-1** es utilizado tanto en el proceso de registro como en el de inicio de sesión, manteniendo consistencia en la implementación.

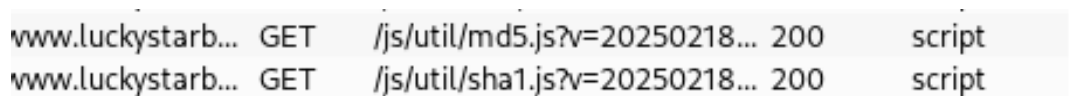


Figura 5: Captura de BurpSuite mostrando las llamadas a las funciones que ejecutan el hash

2.3. Generación del hash desde la consola del navegador

Para identificar las funciones disponibles en el contexto del navegador, se ejecutó el siguiente comando en la consola de desarrollo (F12):

```
for (let k in window) if (typeof window[k] === 'function')  
    console.log(k);
```

```
> for (let k in window) if (typeof window[k] === 'function') console.log(k);  
  
hex_sha1 VM389:1  
b64_sha1 VM389:1  
str_sha1 VM389:1  
hex_hmac_sha1 VM389:1  
b64_hmac_sha1 VM389:1  
str_hmac_sha1 VM389:1  
sha1_vm_test VM389:1  
core_sha1 VM389:1  
sha1_ft VM389:1  
sha1_kt VM389:1  
core_hmac_sha1 VM389:1
```

Figura 6: Lista de funciones disponibles en el objeto `window`, filtradas por “sha”

Una vez identificada la función de hash, se procedió a generar el hash de la contraseña directamente desde la consola del navegador.

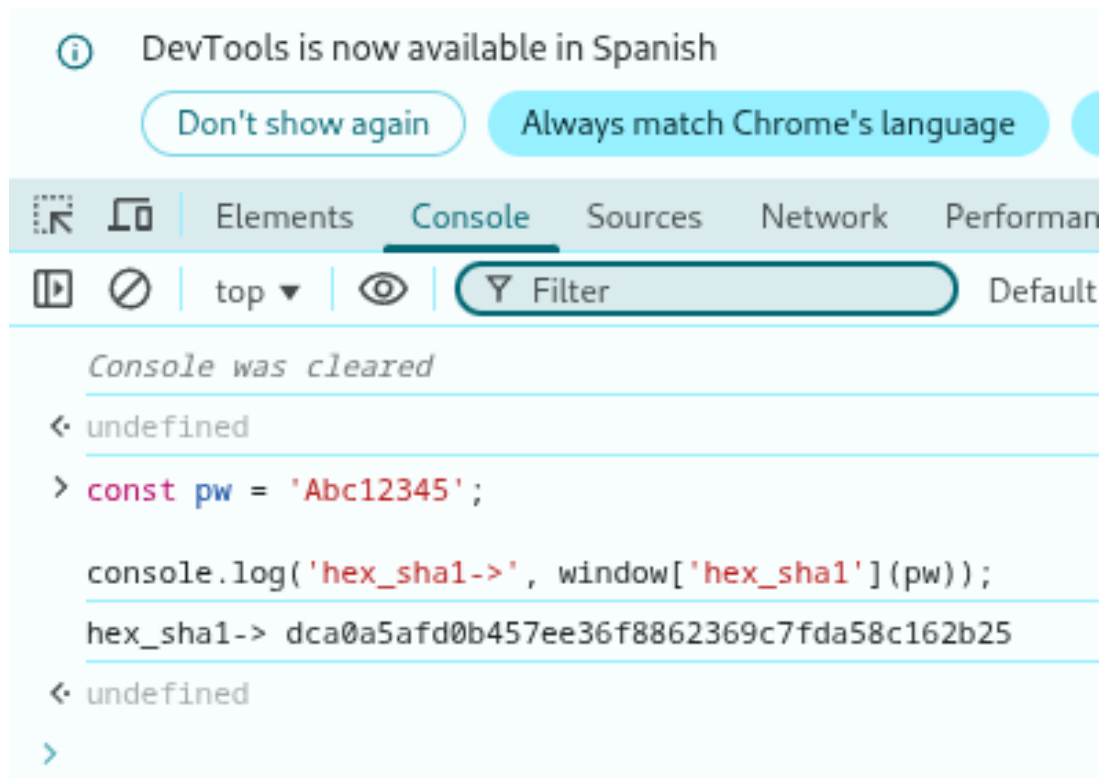


Figura 7: Generación del hash SHA-1 desde la consola, verificando que coincide con el hash enviado en los POST de registro y login

El hash generado manualmente coincidió exactamente con el hash observado en las peticiones de registro e inicio de sesión, confirmando la correcta identificación del algoritmo y su implementación.

Nota: El código JavaScript que implementa la función de hash SHA-1 fue extraído y se encuentra disponible en el repositorio GitHub que acompaña esta entrega con el nombre `sha1.js`.

2.4. Interceptación del tráfico de login con BurpSuite

Utilizando BurpSuite como proxy intermediario, se configuró el navegador para interceptar todo el tráfico HTTP/HTTPS. Durante el proceso de login, se capturaron las peticiones POST dirigidas al endpoint de autenticación.

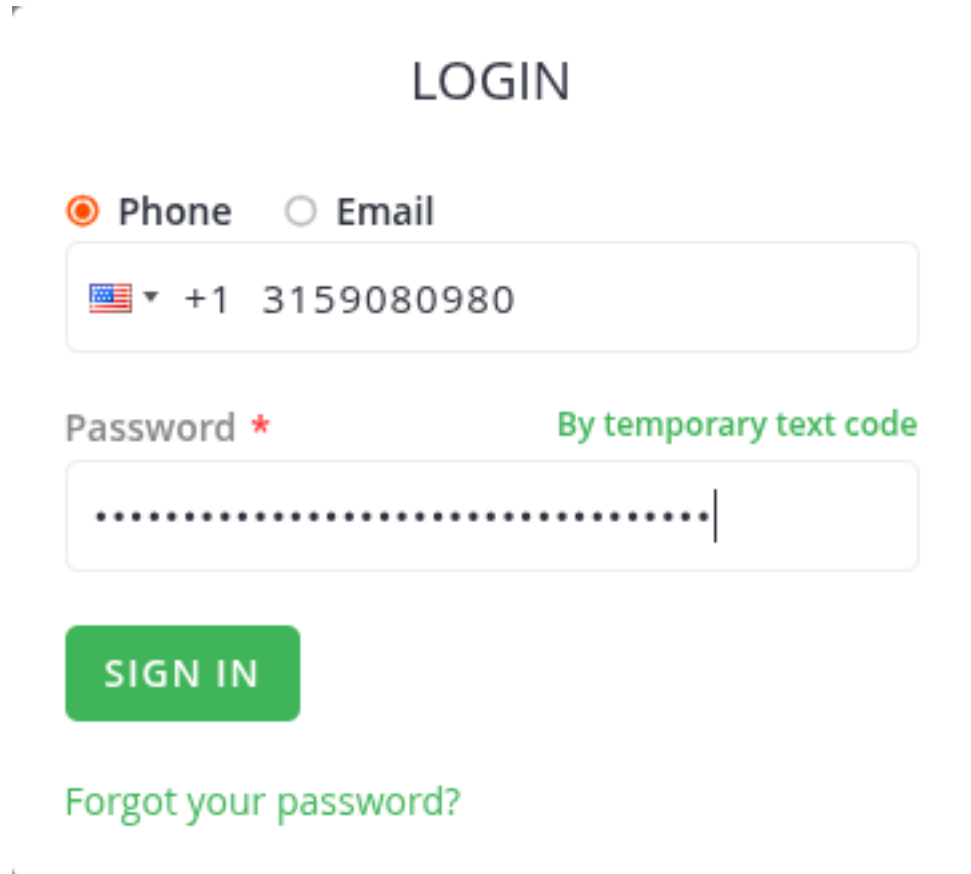
La interceptación permitió:

- Visualizar la estructura completa de las peticiones
- Identificar los parámetros enviados
- Confirmar el uso de SHA-1 en el cliente
- Preparar el entorno para el intento de Pass the Hash

Las figuras 2, 3 y 4 presentadas anteriormente demuestran la capacidad de interceptación exitosa del tráfico.

2.5. Intento de ataque Pass the Hash

Se realizó un intento de ataque Pass the Hash para evaluar si el sistema es vulnerable a este tipo de ataque. El procedimiento consistió en intentar autenticarse utilizando directamente el hash de la contraseña sin conocer la contraseña en texto plano.



LOGIN

☒ Phone ☐ Email

+1 3159080980

Password * [By temporary text code](#)

.....|

SIGN IN

[Forgot your password?](#)

Figura 8: Formulario de inicio de sesión con usuario y contraseña (contraseña oculta)

Limitación crítica identificada: El formulario de contraseña del sitio web tiene una restricción de longitud máxima de 35 caracteres. Dado que un hash SHA-1 genera una cadena de 40 caracteres hexadecimales, **no fue posible realizar el ataque Pass the Hash directamente a través del formulario web.**

Al intentar ingresar una contraseña incorrecta (para posteriormente modificarla con el hash correcto vía BurpSuite), el sistema rechazó el inicio de sesión:

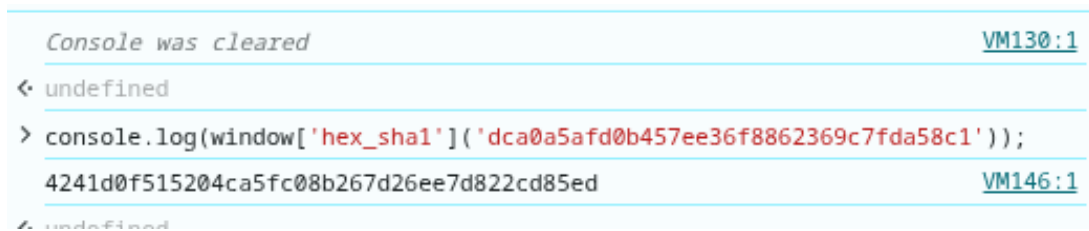
Request		Response			
		Pretty	Raw	Hex	Render
1	HTTP/1.1 200				
2	Server: nginx				
3	Date: Wed, 15 Oct 2025 05:45:54 GMT				
4	Content-Type: application/json; charset=UTF-8				
5	Content-Length: 79				
6	Connection: keep-alive				
7	Access-Control-Allow-Origin: *				
8	Vary: Origin				
9	Access-Control-Expose-Headers: traceId				
10	X-Application-Context: platform-linebus:prod:8033				
11	traceId: 5f0ae74ecc17d27f				
12					
13	{				
	"status":{				
	"code":20016,				
	"msg":"User password invalid"				
	},				
	"result":null,				
	"traceId":5f0ae74ecc17d27f,				

Figura 9: Mensaje de “Invalid password” devuelto por el servidor

Request		Response		
		Pretty	Raw	Hex
8	Access-Token: 5gr4zWc00Z			
9	Sec-Ch-Ua-Mobile: ?0			
10	Tid: 252789338893455872			
11	X-Requested-With: XMLHttpRequest			
12	User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/140.0.0.0 Safari/5			
13	Accept: application/json, text/javascript, */*; q=0.01			
14	Content-Type: application/json			
15	Channelid: 268720169679920128			
16	Sessionid: 0			
17	Origin: https://www.luckystarbus.com			
18	Sec-Fetch-Site: same-origin			
19	Sec-Fetch-Mode: cors			
20	Sec-Fetch-Dest: empty			
21	Referer: https://www.luckystarbus.com/			
22	Accept-Encoding: gzip, deflate, br			
23	Priority: u=1, i			
24	Connection: keep-alive			
25				
26		{ "account": "+13159080980", "password": "4241d0f515204ca5fc08b267d26ee7d822cd85ed", "timestamp": 1760507153759 }		

Figura 10: POST enviado con el login fallido, mostrando el hash del hash de la contraseña

Un hallazgo importante es que cuando se envía el hash como contraseña, **este se vuelve a hashear**. Es decir, el sistema aplica SHA-1 al hash ingresado, generando un “hash del hash”.



```
Console was cleared VM130:1
← undefined
> console.log(window['hex_sha1']('dca0a5afd0b457ee36f8862369c7fda58c1'));
4241d0f515204ca5fc08b267d26ee7d822cd85ed VM146:1
← undefined
```

Figura 11: Demostración mediante la función `hex_sha1`: al agregar el hash de la contraseña menos los últimos 5 dígitos se obtiene el mismo hash del POST login que falló

Esta figura demuestra que el hash enviado en el intento fallido corresponde efectivamente al hash del hash original, confirmando que el sistema aplica doble hashing cuando se intenta un Pass the Hash.

2.6. Políticas de privacidad y seguridad

Privacy Policy

Information such as name, e-mail address, mailing address, phone number or credit card number used during purchase of a Lucky Star E-Ticket will never be used or sold to third party vendors for any Lucky Star benefit.

Information input to our website to purchase E-tickets or make a Frequent Rider Membership is provided by the customer and is used to improve customer service inquiries such as:

- Helping you retrieve an itinerary
- Confirming a transaction

All personal information input for purchase is encrypted in our servers and is only accessible to those authorized to do so.

Figura 12: Políticas de seguridad LuckyStar

Se revisaron las políticas de privacidad y seguridad del sitio web auditado, las cuales se encuentran disponibles en el pie de página del portal. El sitio utiliza el protocolo HTTPS para todas las comunicaciones, garantizando el cifrado de los datos durante la transmisión. No obstante, las políticas no especifican con claridad si las contraseñas se almacenan de forma segura en el servidor mediante un proceso de hashing, o si simplemente se aplica un hash durante el envío. Tampoco se menciona explícitamente el uso de SHA-1 ni de ningún otro algoritmo de cifrado o integridad de datos en la documentación pública.

La política de privacidad declara que la información personal como nombre, correo electrónico, dirección, número telefónico o datos de tarjeta de crédito— no será vendida ni compartida con terceros, y que los datos ingresados para la compra de boletos o membresías son cifrados y accesibles solo para personal autorizado. Si bien esto refleja

un compromiso básico con la protección de la información del usuario, persiste una falta de transparencia técnica respecto a las prácticas de seguridad concretas implementadas. En particular, no se detalla cómo se gestionan las contraseñas ni qué mecanismos existen para prevenir accesos indebidos, lo que constituye una debilidad relevante en términos de confianza y cumplimiento de estándares actuales de seguridad web.

2.7. Conclusiones sobre la seguridad del sitio

Tras el análisis exhaustivo de la implementación de hash en el sitio web encontrado, se presentan las siguientes conclusiones:

2.7.1. Conclusión 1: Uso de SHA-1, un algoritmo obsoleto

El sitio utiliza **SHA-1** para hashear contraseñas, un algoritmo que ha sido considerado criptográficamente inseguro desde 2017. SHA-1 es vulnerable a ataques de colisión y no debe utilizarse para aplicaciones de seguridad críticas como el almacenamiento de contraseñas. Se recomienda migrar a algoritmos más robustos como **bcrypt**, **scrypt**, **Argon2** o **PBKDF2**, que están específicamente diseñados para el hashing de contraseñas e incluyen factores de trabajo ajustables y salt automático.

2.7.2. Conclusión 2: Hashing del lado del cliente como falsa sensación de seguridad

Aunque hashear las contraseñas en el lado del cliente antes de transmitir las puede parecer una medida de seguridad adicional, en la práctica presenta serias limitaciones. El hash se convierte efectivamente en la contraseña, ya que es lo que el servidor espera recibir. Si un atacante intercepta el hash (mediante un ataque man-in-the-middle o accediendo a los datos del servidor), puede utilizarlo directamente para autenticarse sin necesidad de conocer la contraseña original. Esto convierte al hash en un objetivo valioso para los atacantes.

2.7.3. Conclusión 3: Protección parcial contra Pass the Hash mediante limitación de caracteres

La restricción de 35 caracteres en el campo de contraseña ofrece una protección parcial frente a ataques Pass the Hash directos, ya que impide ingresar un hash SHA-1 completo de 40 caracteres. Sin embargo, esta limitación no constituye una medida de seguridad efectiva. Un atacante podría modificar el código JavaScript del cliente o interceptar y alterar el tráfico mediante herramientas como BurpSuite. Además, al tratarse de una restricción aplicada únicamente del lado del cliente, puede ser fácilmente eludida mediante el envío de una solicitud POST modificada.

2.7.4. Conclusión 4: Falta de implementación de medidas de seguridad modernas

El análisis reveló la ausencia de varias medidas de seguridad que hoy se consideran estándar. En primer lugar, las contraseñas se almacenan utilizando SHA-1 sin la incorporación de un salt, lo que las deja expuestas a ataques de rainbow tables y permite identificar fácilmente a los usuarios que comparten contraseñas idénticas. Tampoco se observaron mecanismos visibles de limitación de intentos de inicio de sesión, lo que aumenta

el riesgo frente a ataques de fuerza bruta. Además, el sistema no ofrece autenticación de dos factores (2FA), una capa adicional que suele ser esencial para proteger el acceso a las cuentas. Finalmente, las políticas de seguridad presentan un nivel de transparencia limitado, pues no detallan de forma clara las medidas implementadas, dificultando que los usuarios puedan evaluar el grado real de protección de sus datos.

3. Referencias y Materiales

Todos los archivos de evidencia mencionados en este informe están disponibles en el repositorio GitHub adjunto a esta entrega:

- `sha1.js` - Código JavaScript extraído que implementa el hash SHA-1
- Capturas de pantalla de BurpSuite y consola del navegador